

Complete Skills Inventory

Foundation (Python, Math, Data)

Python (advanced, async, OOP, decorators, generators) | Pandas, NumPy | SQL and advanced SQL (window functions, CTEs, query optimization) | PostgreSQL, MongoDB | Basic statistics and probability | Linear algebra (matrix ops, dot products, eigenvalues) | ORM (SQLAlchemy)

Core ML

Gradient descent, loss functions, backpropagation | Classification, regression, clustering | Train/test splits, cross-validation, overfitting | PyTorch (primary framework) | Feature engineering | Model evaluation metrics

LLM Fundamentals

Transformer architecture and attention mechanisms | GPT-style training (pre-training, RLHF, DPO) | Tokenization (BPE, SentencePiece) | Embeddings and vector representations | Context windows, token limits

Working With LLMs

OpenAI / Anthropic APIs | Prompt engineering (zero-shot, few-shot, CoT, structured output) | Function calling / Tool use | Model Context Protocol (MCP) | Hugging Face Transformers ecosystem | Fine-tuning (LoRA, PEFT, QLoRA) | Modern evaluation (LLM-as-judge, custom rubrics, human eval)

NLP

Sentiment analysis (aspect-based, emotion detection) | Named Entity Recognition (NER) | Intent detection and text classification | Text summarization (extractive and abstractive) | Keyword/topic extraction | Dialogue management and state tracking | spaCy, Hugging Face NLP pipelines

RAG & AI Applications

Vector databases (Pinecone, ChromaDB, Weaviate, pgvector) | Embedding strategies and chunking | Hybrid search (vector + keyword) | RAG pipeline architecture (retrieval, re-ranking, generation) | LangChain, LlamaIndex | Direct SDK usage (frameworkless RAG) | RAG evaluation (faithfulness, relevance, groundedness) | FastAPI for AI serving

Multimodal AI

Vision-Language Models (GPT-4V, Claude Vision, Gemini) | Image generation (DALL-E, Stable Diffusion APIs) | Audio/Video understanding (Whisper, transcription) | Multimodal RAG

AI Agents

ReAct pattern, planning-execution loops | Agent memory (short-term, long-term, episodic) | Multi-agent coordination and orchestration | LangGraph, CrewAI | Human-in-the-loop patterns | Agent evaluation and debugging

AI Safety & Production

Prompt injection defense | Output filtering and guardrails | Hallucination mitigation | Data privacy in AI pipelines | Rate limiting and cost management | Responsible AI (bias, fairness, transparency)

AI in Production Infrastructure

AI as a microservice (sync/async/streaming patterns) | gRPC + AI serving | Semantic caching and optimization | LLMOps (LangSmith, Weights & Biases) | Feature flags for AI rollout | AI gateway patterns (multi-provider routing) | Docker, Kubernetes for AI workloads | CI/CD for ML pipelines | AWS/GCP for AI (Lambda, SageMaker, Vertex AI, Cloud Functions) | Serverless AI deployment

Data Engineering (AI-relevant)

Snowflake, BigQuery | Data pipelines for ML (dbt, Spark basics) | Dimensional data modeling | Automated ML pipelines

Soft Skills

Breaking down business problems into AI solutions | Cross-functional collaboration | End-to-end ownership (dev to production) | AI governance, security, compliance

Learning Roadmap — Go In Order

Think of this as building a house. Each stage depends on the one before it. Don't skip ahead — even stages you think you know. Use them as a recap if nothing else.

STAGE 1: The Foundation — Python, SQL, Math

Duration: 3-5 weeks (recap mode) or 8-10 weeks (learning fresh)

Everything else depends on this. Even if you write Python daily, ML-style Python is different — heavy NumPy/Pandas, vectorized operations, data manipulation patterns. Revisit this to build the muscle memory.

- **Python (advanced)** — Functions, classes, decorators, generators, async programming, error handling, type hints
 - Focus on: list comprehensions, lambda, map/filter, context managers — these appear everywhere in ML code
- **Pandas** — DataFrames, groupby, merge, pivot, handling missing data, time series operations
 - This is your primary data manipulation tool for any ML project
- **NumPy** — Array operations, broadcasting, matrix multiplication, random number generation
 - Understand vectorized operations — they're 100x faster than Python loops for numerical work
- **SQL (advanced)** — Joins, subqueries, window functions (ROW_NUMBER, RANK, LAG/LEAD), CTEs, query optimization
 - Window functions and CTEs are critical for feature engineering from databases
- **PostgreSQL and MongoDB** — When to use relational vs document stores, indexing strategies, JSON operations in Postgres
- **Basic statistics** — Mean, median, standard deviation, distributions (normal, Poisson), probability, Bayes' theorem
 - You don't need a stats degree — just enough to understand what models are doing and why
- **Linear algebra essentials** — Vectors, matrices, dot products, matrix multiplication, transpose, eigenvalues/eigenvectors
 - This is the language neural networks speak — every layer is a matrix multiplication
- **ORM (SQLAlchemy)** — Model definitions, queries, relationships, migrations — connecting Python to databases cleanly

Resources: Python.org docs, W3Schools SQL, Khan Academy (statistics + linear algebra), 3Blue1Brown 'Essence of Linear Algebra' series

Recap Project: Load a dataset of 10K+ business reviews into PostgreSQL, write advanced SQL queries (window functions to rank businesses by sentiment trend over time), then replicate the same analysis in Pandas. Compare approaches. This bridges your existing SQL skills with the Pandas patterns you'll use throughout ML.

STAGE 2: Machine Learning Fundamentals

Duration: 6-8 weeks

Before you touch any LLM, understand what machine learning actually is. This stage teaches you how models learn, why they fail, and how to evaluate them. Skip this and you'll be debugging black boxes forever.

- **How models learn** — Gradient descent (batch, stochastic, mini-batch), loss functions (MSE, cross-entropy), learning rates
 - Implement gradient descent from scratch in NumPy before using any framework — this builds real intuition
- **Backpropagation** — Chain rule, computational graphs, how gradients flow through a network
 - Watch 3Blue1Brown's neural network series — best visual explanation available
- **Classification** — Logistic regression, decision trees, random forests, SVMs — understand the classics
 - Build a spam classifier and a customer churn predictor as practice
- **Regression** — Linear regression, polynomial regression, regularization (L1/L2/ElasticNet)
 - Predict something real — pricing, engagement metrics, review ratings
- **Clustering** — K-means, DBSCAN, hierarchical clustering — unsupervised pattern finding
 - Useful for customer segmentation, topic grouping in reviews
- **Train/test splits and cross-validation** — Why you can't evaluate on training data, k-fold cross-validation, stratified splitting
- **Overfitting and regularization** — Dropout, early stopping, L1/L2 penalties — recognizing and fixing overfit models
- **Evaluation metrics** — Accuracy, precision, recall, F1-score, AUC-ROC, confusion matrices
 - Different metrics matter for different problems — learn when to use which
- **PyTorch fundamentals** — Tensors, autograd, nn.Module, DataLoader, training loops, GPU acceleration
 - Build a complete neural network from scratch — MNIST digit classifier is the classic starting point
- **Feature engineering** — Numerical scaling, categorical encoding, text features, feature selection, handling imbalanced data

Resources: fast.ai Practical Deep Learning (free, excellent), Andrej Karpathy's YouTube channel, Kaggle Learn courses

Project: Build a review sentiment classifier in PyTorch — load a review dataset, engineer features, train a model, evaluate with proper metrics, handle class imbalance. Deploy it as a simple FastAPI endpoint. Directly relevant to Reputation Management.

STAGE 3: How LLMs Actually Work

Duration: 4-6 weeks

This is where it gets exciting. Understanding transformer internals separates engineers who debug AI issues from those who stare at black boxes. Don't rush this — it pays dividends in every stage after.

- **The Transformer architecture** — Encoder, decoder, encoder-decoder variants — understand the full picture
 - Read 'Attention Is All You Need' after watching visual explanations, not before
- **Self-attention mechanism** — Query-Key-Value, scaled dot-product attention, why attention works
 - Jay Alammar's 'The Illustrated Transformer' is the best entry point
- **Multi-head attention** — Why multiple attention heads matter, what different heads learn, concatenation and projection
- **Positional encoding** — How transformers handle sequence order — sinusoidal, learned, RoPE, ALiBi
- **How GPT-style models are trained** — Pre-training on next-token prediction, massive datasets, compute requirements
- **RLHF and DPO** — How models go from 'predict next token' to 'follow instructions helpfully' — reinforcement learning from human feedback, Direct Preference Optimization
- **Tokenization deep dive** — BPE, SentencePiece, WordPiece — why tokenization choices affect model behavior
 - Especially important for non-English text, code, and domain-specific vocabulary
- **Embeddings** — Word embeddings (Word2Vec, GloVe history), sentence embeddings, how semantic similarity works under the hood
- **Context windows and limitations** — Token limits, attention scaling (quadratic cost), why models hallucinate, lost-in-the-middle problem

Resources: Karpathy's 'Let's build GPT from scratch' (YouTube, gold standard), Jay Alammar's blog, 3Blue1Brown's attention video

Project: Implement a minimal GPT from scratch following Karpathy's tutorial. Train it on a small text corpus. Then experiment — change attention heads, context size, embedding dimensions. See how each change affects generation quality.

STAGE 4: Working With LLMs — APIs, Prompting, and Modern Patterns

Duration: 6-8 weeks

Now you start using LLMs, not just understanding them. The original roadmap covered API calls and basic prompting. This version adds function calling, MCP, and modern evaluation — the patterns that actually matter in production.

- **OpenAI API** — Messages API, system/user/assistant roles, streaming, temperature/top-p, structured output (JSON mode)
- **Anthropic API** — Messages API, system prompts, multi-turn conversations, streaming, extended thinking
 - Get hands-on with both — different strengths, and you'll use both professionally
- **Prompt engineering (foundational)** — Zero-shot prompting — give instructions, get results without examples

- **Prompt engineering (intermediate)** — Few-shot prompting — provide examples to guide model behavior
- **Prompt engineering (advanced)** — Chain-of-thought, self-consistency, tree-of-thought, prompt chaining
- **Structured output** — JSON mode, schema enforcement, output parsing — getting reliable structured data from LLMs
- **Function calling / Tool use** — The model decides when to call your APIs — request format, function definitions, execution flow
 - This is how modern AI integrations work. Critical for any Vendasta AI feature
- **Model Context Protocol (MCP)** — The emerging standard for connecting AI to external tools and data sources
 - You already use MCP at work through Claude — now understand the protocol deeply
- **Hugging Face ecosystem** — Model Hub, Transformers library, loading models locally, model cards, tokenizers, pipelines
- **Fine-tuning with LoRA/PEFT** — When prompting isn't enough — adapter-based fine-tuning on domain-specific data
- **QLoRA** — Quantized fine-tuning — fine-tune large models on consumer hardware
- **Modern evaluation frameworks** — LLM-as-judge, custom rubrics, human evaluation pipelines, A/B testing prompts
 - Legacy: BLEU/ROUGE exist but are outdated for LLM evaluation — know them for historical context only

Resources: Anthropic docs (prompt engineering guide, function calling), OpenAI cookbook, Hugging Face NLP course (free), DeepLearning.AI short courses

Project: Build a review response generator that uses function calling to fetch business context (hours, services, location) from a mock API, then generates personalized, context-aware replies. Implement a custom evaluation pipeline using LLM-as-judge to measure response quality. Relevant across Reputation Management and CRM.

STAGE 5: NLP for Product Engineers

Duration: 4-6 weeks | NEW — not in original roadmap

The original roadmap barely mentioned NLP. But at Vendasta, almost every product deals with text — reviews, social posts, listing descriptions, customer messages, search queries. NLP is the bread and butter.

- **Sentiment analysis** — Beyond positive/negative — aspect-based sentiment ('food was great, service was slow'), emotion detection, sarcasm handling
 - Use both classical (spaCy) and LLM-based approaches — compare trade-offs
- **Named Entity Recognition (NER)** — Extract business names, locations, services, competitor mentions from unstructured text
 - Custom NER training for domain-specific entities (e.g., extracting service types from reviews)
- **Text classification** — Categorize reviews by topic, flag urgent issues, route customer messages to right team
- **Intent detection** — What does the customer actually want? Map natural language to actions — critical for chatbots and CRM
- **Dialogue management and state tracking** — Multi-turn conversation flows, maintaining context, slot filling
 - Relevant for building any conversational AI feature across Vendasta products
- **Text summarization** — Extractive (pick key sentences) vs. abstractive (rewrite in new words) — when to use which
 - Summarize review trends, social engagement reports, customer feedback themes
- **Keyword and topic extraction** — TF-IDF, TextRank, LLM-based extraction — what terms are customers using?
 - Directly applicable to Local SEO — identifying what keywords to target from real customer language
- **spaCy** — Industrial-strength NLP — tokenization, POS tagging, dependency parsing, NER, custom pipelines

Resources: Hugging Face NLP course, spaCy docs and tutorials, Stanford NLP (CS224N) lectures on YouTube

Project: Build a 'Review Intelligence Pipeline' — ingest 5K+ reviews, run sentiment analysis (aspect-level), extract entities (business names, services, locations), classify by topic, and generate a summary dashboard. Serve results via API. Relevant to Reputation Management, Local SEO, and CRM.

STAGE 6: RAG Pipelines & AI Applications

Duration: 6-8 weeks

This is where your full-stack background becomes a superpower. Most ML engineers struggle to build production apps — you already ship software daily. Now you add AI to the mix.

- **Vector databases deep dive** — Pinecone (managed, scalable), ChromaDB (local, fast prototyping), Weaviate (open-source), pgvector (Postgres extension)
 - Understand trade-offs: managed vs self-hosted, cost, latency, filtering capabilities
- **Embedding models and strategies** — OpenAI embeddings, Sentence Transformers, Cohere Embed — choosing the right model for your data
- **Chunking strategies** — Fixed-size, semantic, recursive, document-structure-aware — chunk size affects retrieval quality dramatically
- **Hybrid search** — Vector similarity + keyword search (BM25) combined — better retrieval than either alone
- **RAG pipeline architecture** — Full pipeline: ingest -> chunk -> embed -> store -> retrieve -> re-rank -> assemble context -> generate
 - Understand each step deeply — most RAG failures happen at retrieval, not generation
- **Re-ranking** — Cross-encoder re-ranking, Cohere Rerank, ColBERT — improving retrieval precision after initial search
- **LangChain** — Chains, memory management, document loaders, output parsers — good for prototyping
 - Know it, but don't depend on it — the trend is moving toward lighter solutions
- **LlamaIndex** — Data connectors, index types, query engines — designed specifically for RAG over your data
- **Direct SDK usage (frameworkless)** — Build RAG using raw Anthropic/OpenAI SDKs — understand what frameworks abstract away
 - This is the industry trend: simpler, more controllable, easier to debug
- **RAG evaluation** — Faithfulness (did it make stuff up?), relevance (did it find the right docs?), groundedness metrics
- **FastAPI for AI serving** — Streaming responses, async endpoints, proper error handling, request validation for AI APIs

Resources: Anthropic RAG cookbook, Pinecone learning center, LlamaIndex docs, Jerry Liu's blog posts

Project: Build a 'Local SEO Advisor' — a RAG system that ingests a business's reviews, listings, and competitor data, then answers questions like 'What keywords should I target?' or 'What are my customers complaining about most?' Serve it via a REST/gRPC API with streaming responses.

STAGE 7: Multimodal AI

Duration: 4-6 weeks | NEW — not in original roadmap

The original roadmap completely ignored multimodal AI. For Social Marketing and any content-heavy product, this is arguably more important than text-only LLM skills. Images, video, and audio are everywhere.

- **Vision-Language Models** — GPT-4V, Claude Vision, Gemini — analyze images, extract text from screenshots, understand visual content
 - Use case: analyze social media post images, extract info from review screenshots, competitor visual analysis
- **Image generation APIs** — DALL-E 3, Stable Diffusion, Midjourney — generate marketing visuals, social media graphics
 - Focus on API usage and prompt engineering for images, not training your own models
- **Image understanding for products** — Auto-tagging social media images, visual content moderation, brand logo detection
- **Audio/Video understanding** — Whisper for transcription, video summarization, social video content analysis
 - Transcribe customer video reviews, analyze social video engagement
- **Multimodal RAG** — Combine text and image retrieval — search across documents that contain both text and visuals
 - Think: product catalogs, marketing materials, mixed-media content libraries
- **Multimodal embeddings** — CLIP, Nomic Embed Vision — encode images and text into the same vector space for cross-modal search

Resources: OpenAI Vision docs, Anthropic multimodal guide, Hugging Face multimodal models, Stable Diffusion docs

Project: Build a 'Social Content Generator' — given a business context and topic, generate a social media post with both text and an AI-generated image. Add vision analysis to score existing posts for visual quality and engagement potential.

STAGE 8: AI Agents & Orchestration

Duration: 6-8 weeks

Agents are the most active area in AI right now. The original gave this 1-2 months with surface coverage. This expanded version goes deep — agents will be the primary way AI features work in production SaaS.

- **Agent fundamentals** — What agents are — LLMs that can reason, plan, and take actions in the real world
- **ReAct pattern** — Reasoning + Acting — the foundational agent loop: think -> decide tool -> execute -> observe -> repeat
- **Building agents from scratch** — Don't start with frameworks — build a simple agent using raw API calls + function calling first
 - This builds real understanding of what agent frameworks abstract away
- **Tool use in agents** — Defining tools, letting the agent choose which to use, handling tool errors, tool composition
- **Agent memory** — Short-term (conversation context), long-term (vector store), episodic (past experience summaries)
 - Memory design determines whether your agent is useful or forgetful

- **Planning and execution** — Task decomposition, step-by-step planning, self-correction, backtracking on failures
- **LangGraph** — Graph-based agent orchestration — nodes, edges, state, conditional routing, human-in-the-loop
- **CrewAI** — Role-based multi-agent systems — define agent roles, tasks, and collaboration patterns
- **Multi-agent coordination** — Orchestrator patterns, agent handoffs, shared state, conflict resolution between agents
- **Human-in-the-loop patterns** — When the agent should ask for confirmation vs. act autonomously — critical for production
 - Get this wrong and your agent either annoys users or does dangerous things unsupervised
- **Agent evaluation and debugging** — Tracing agent decisions, measuring task completion rates, handling infinite loops, cost monitoring

Resources: Anthropic's 'Building Effective Agents' guide, LangGraph docs, Andrew Ng's agent courses on DeepLearning.AI, Harrison Chase's talks

Project: Build a 'Social Marketing Agent' that can: (1) analyze a business's recent post performance via tools, (2) research trending topics in their industry, (3) generate post text + image, (4) present to user for approval, (5) schedule upon approval. Multi-step agent with human-in-the-loop before any irreversible action.

STAGE 9: AI Safety, Security & Responsible AI

Duration: 4-6 weeks | NEW — not in original roadmap

The original roadmap had zero coverage of AI safety. If you're shipping AI features to real customers handling real business data at Vendasta, this isn't optional — it's a professional responsibility.

- **Prompt injection attacks** — Direct injection, indirect injection, jailbreaking — how attackers manipulate AI through crafted inputs
 - Learn to attack your own systems so you can defend them
- **Prompt injection defense** — Input sanitization, output filtering, system prompt hardening, canary tokens, separation of data and instructions
- **Output guardrails** — Content moderation, PII detection and redaction, preventing harmful/off-topic responses
 - NeMo Guardrails, Anthropic's constitutional AI approach, custom filter chains
- **Hallucination mitigation** — Grounding techniques, citation verification, confidence scoring, retrieval-augmented generation as defense
 - The #1 complaint about AI in production — learn to minimize it systematically
- **Data privacy in AI pipelines** — Customer data in prompts, embedding storage security, data retention policies, GDPR/compliance
 - Vendasta handles SMB data — this is critical for customer trust
- **Rate limiting and cost management** — Token budgets per user/request, caching strategies, model fallbacks (expensive model -> cheap model)
 - AI features that cost \$50/user/month won't ship — learn to optimize ruthlessly
- **Monitoring and observability** — LangSmith for tracing, Weights & Biases for experiments, custom logging for production AI
 - You need to know when your AI is misbehaving before customers tell you
- **Responsible AI** — Bias detection in training data and outputs, fairness testing across demographics, transparency in AI-generated content
- **AI governance and compliance** — Documentation standards, audit trails, model versioning, regulatory awareness

Resources: OWASP LLM Top 10, Anthropic's responsible AI docs, NeMo Guardrails (NVIDIA), Simon Willison's blog on prompt injection

Project: Take any AI project you built in previous stages and harden it — add prompt injection defense, output guardrails, PII redaction, rate limiting, cost tracking, and monitoring. Write a security review document. This is the difference between a demo and a production feature.

STAGE 10: AI Integration in Microservice Architecture

Duration: 4-6 weeks | NEW — not in original roadmap

This stage makes everything click for your role at Vendasta. You already build microservices with Go and gRPC — now learn the specific patterns for integrating AI into that architecture. This is where you become uniquely valuable.

- **AI as a microservice** — Design patterns — when AI should be its own service vs. embedded in existing services
 - Sync vs async processing, streaming responses, queue-based batch processing
- **gRPC + AI serving** — Streaming token-by-token responses over gRPC, proto design for AI endpoints, bidirectional streaming
 - This maps directly to Vendasta's architecture — you'll be designing these protos
- **Semantic caching** — Cache similar queries, not just identical ones — reduce latency and API costs dramatically
- **Model routing and fallbacks** — Route to different models based on task complexity, automatic fallback if primary model is down
- **AI gateway patterns** — Centralized AI routing service — model selection, load balancing, rate limiting, logging, cost tracking
 - One gateway serving multiple Vendasta products, each with different AI needs
- **Feature flags for AI** — Gradually rolling out AI features, A/B testing AI vs. non-AI flows, kill switches for when things go wrong
- **Observability for AI services** — Distributed tracing across AI calls, monitoring token usage per service, alerting on quality degradation
- **CI/CD for AI** — Testing AI endpoints (deterministic + fuzzy testing), staging environments with model versioning, canary deploys
- **Docker and Kubernetes for AI** — GPU-aware scheduling, model loading strategies, horizontal scaling for inference
- **Cloud AI services** — AWS SageMaker, GCP Vertex AI, managed endpoints vs. self-hosted — when to use which
- **Serverless AI** — Lambda/Cloud Functions for lightweight inference, cold start considerations, timeout management

Resources: Chip Huyen's 'Designing ML Systems', Anthropic production best practices, MLOps Community, Google's MLOps maturity model

Capstone Project: Design and build a complete AI microservice for Vendasta's architecture — a 'Business Intelligence Agent' that ingests data from multiple product APIs (reviews, social posts, SEO metrics), runs analysis using RAG + NLP, and serves actionable insights via gRPC endpoints. Include: streaming responses, semantic caching, rate limiting, prompt injection defense, monitoring, feature flags, and multi-model fallback. Ship-ready.

STAGE 11: Data Engineering for AI (Ongoing)

Duration: Ongoing — learn as needed

Not a dedicated study phase — pick these up as your AI projects demand more sophisticated data handling. Relevant when you're working with large-scale data pipelines that feed AI systems.

- **Snowflake / BigQuery** — Analytical databases for large-scale data — ML feature stores, training data management
- **Data pipelines for ML** — dbt for transformation, data quality checks, lineage tracking
- **Spark basics** — Distributed processing for when your data doesn't fit in memory — PySpark for ML preprocessing
- **Dimensional data modeling** — Star schema, fact/dimension tables — organizing data for efficient ML feature extraction
- **Automated ML pipelines** — End-to-end automation: data ingestion -> preprocessing -> training -> evaluation -> deployment

Resources: dbt docs, Snowflake University (free), Google BigQuery sandbox, PySpark tutorials

Complete Timeline Summary

Stage	Focus	Duration	Status
1	Python, SQL, Math Foundations	3-5 wks*	Kept
2	Machine Learning Fundamentals	6-8 wks	Kept
3	How LLMs Actually Work	4-6 wks	Kept
4	Working With LLMs + Modern Patterns	6-8 wks	Expanded
5	NLP for Product Engineers	4-6 wks	New
6	RAG Pipelines & AI Applications	6-8 wks	Expanded
7	Multimodal AI	4-6 wks	New
8	AI Agents & Orchestration	6-8 wks	Expanded
9	AI Safety, Security & Responsible AI	4-6 wks	New
10	AI in Microservice Architecture	4-6 wks	New
11	Data Engineering for AI	Ongoing	Kept
	Total (excluding Stage 11)	~12-17 months	

* Stage 1 is 3-5 weeks in recap mode (your case) or 8-10 weeks if learning fresh.

Final Thought: This roadmap is designed to be two things at once — a learning path you follow in order, and a reference document you come back to in 2028 when you need to remember how RAG evaluation works or what ReAct agents are. Every section includes enough detail to jog your memory, not just a bullet point you'll forget. The projects are designed to compound — each one builds on skills from previous stages and maps to a real Vendasta product. By Stage 10, you're not just an AI/ML practitioner — you're the person who can look at any Vendasta product and say 'I know exactly where AI fits here, and I can build it.'